Falcon user guide

PDF and screenshot exports

- **PDFs** of this Markdown guide (per page + one combined file) are under `export/pdf/`.
- **PNG screenshots** of the **auth flow in the apps** (demos, hosted Falcon Auth, optional console) are under `export/screenshots/auth-flow/`.

Regenerate everything from `export/README.md` with `bun run export` in `docs/user-guide/export/`.

This guide explains Falcon for people who **use**, **integrate**, or **operate** the platform: product owners, support, and application developers. It covers Falcon Auth (centralized identity), Falcon Connect (app-to-app connections), the web console, and the JavaScript SDK.

How to read this guide

- Start with What is Falcon? and Architecture if you are new.
- **Building an app that signs users in?** See Falcon Auth, especially Centralized sign-in and App registration.
- **Connecting your product to another Falcon-registered app?** See Falcon Connect.
- **End users in the browser?** See Using the console.

Contents

Getting started

Document	Description
What is Falcon?	Products, audiences, and terminology
Architecture	Services, data stores, and request flows

Falcon Auth

Document	Description
Falcon Auth overview	Section index
Centralized sign-in	Hosted UI, redirects, and return URLs
Registering client apps	Database registration, origins, and keys
Sessions and cookies	How sessions work across apps
Organizations	Orgs, members, and Better Auth plugin

Falcon Connect

Document	Description
Falcon Connect overview	Section index
Concepts	Apps, capabilities, scopes, org context
Installation requests and approval	End-to-end connection flow
Managing connections	Status, pause, resume, revoke, sync
Calling the Connect API	Headers, session vs JWT, organizations

Console

Document	Description
Console overview	Section index
Account, profile, and connected apps	User-facing console features
Organizations in the console	Creating orgs and switching context
Connections in the console	Viewing and managing connections

SDK and integration

Document	Description
SDK overview	Section index
React: provider, hooks, components	`FalconAuthProvider`, hooks, optional UI
Hosted sign-in URLs	Building redirect links
Server session verification	`verifySession` for backends
Connect display helpers	Optional UI utilities for connection lists

Reference

Document	Description
Reference overview	Section index
Environment variables	Typical configuration by component
Permissions matrix	Connect actions by org role

Documentation version: aligned with the repository's Falcon Auth hosted flow, Connect API v1, and console features. For low-level API schemas, see the generated OpenAPI or RPC reference where your deployment exposes it.

Account, profile, and connected apps

The console's **account** area is where a signed-in user reviews their **profile** and manages aspects of their relationship with Falcon.

Profile

Users can see their **name**, **email**, and account metadata (for example member since). Name updates flow through the same Better Auth client the console uses elsewhere; exact fields depend on your Better Auth configuration.

Connected Falcon Auth apps

The console lists **applications you have signed into** through Falcon Auth (rows linked via `app_user`). For each entry you typically see:

- **Application name** (from `falcon_auth_app.name`).
- **When** the link was created (connected timestamp).

Revoking access

Users may **revoke** their link to an app. That removes the association for future directory purposes; the third-party app may still hold its own data and should handle "account disconnected" in its product logic if required.

This control is **user-centric** (like "remove this app from my account"), not the same as revoking a **Connect** relationship between two products (that lives under Connections).

SDK documentation surface

The console may embed a **developer-focused** page (installation, provider, components, hooks, server helpers, registration notes). That content targets **integrators** who ship code against `@falcon-framework/sdk`.

End-user documentation for “how to use my company’s portal” usually lives separately from SDK API details; this **user guide** is the narrative complement.

Sign-in experience

The console signs in **directly** against the auth server from its own origin (embedded login in the console SPA). Your customer-facing apps may instead use **hosted** sign-in; both patterns share the same user database when pointed at the same `BETTER_AUTH_URL`.

See Centralized sign-in.

Related topics

- Organizations
- React integration

Connections in the console

When the console is configured with a **Connect service** base URL, authenticated users can inspect **Falcon Connect** data for their **active organization**.

Connection list

The list view groups connections by **status**:

- **Active** — operational integrations.
- **Paused** — temporarily disabled.
- **Revoked** — terminated; should not be used for new operations.

Each row identifies participating **Connect apps** by name (resolved from the app directory) and shows metadata such as creation time.

Connection detail

Opening a connection shows:

- **Status** and scope summary.
- **Actions** the user is allowed to perform (subject to role), such as **pause**, **resume**, **revoke**, or **request sync**—matching the API described in Managing connections.

Permissions

Console buttons call the same Connect endpoints your apps use. If the user's role is insufficient (for example **member** trying to revoke), the API returns **403** and the UI surfaces an error.

Relationship to installation requests

Some product flows create **installation requests** outside the console (from a source app). The console is still a valid place to **monitor** resulting connections once approved.

For the cross-app browser handoff pattern, see Installation requests and approval.

Related topics

- Falcon Connect overview
- Account and connected apps

Organizations in the console

Organizations are the **tenant boundary** for Falcon Connect and the **membership** model for collaboration in the console.

Creating an organization

Users open the **create organization** flow, provide a **name**, and usually a **slug** (URL-friendly identifier). On success, Better Auth returns the new organization id.

The console then **sets the active organization** so subsequent Connect API calls include the correct `X-Organization-Id`.

Switching organizations

Users with multiple memberships can **switch** the active organization (org switcher in the shell). Switching updates:

- Which **connections** and **installation requests** load from Connect.
- Which **role** applies for permission checks (roles are per membership).

Roles at a glance

Role	Typical use
owner	Full control; can approve installations; manage connections.
admin	Same as owner for Connect permission checks in the stock matrix.
member	May create installation requests; cannot approve or revoke connections.

Exact strings must match what Better Auth stores and what Connect checks. See Permissions matrix.

Why orgs matter for Connect

Without an active org (or with zero memberships), the console cannot meaningfully call `/v1/*` routes: Connect refuses ambiguous organization context.

Demo applications mirror this by redirecting users to **org creation** when they have no orgs.

Related topics

- Falcon Auth organizations
- Calling the Connect API

Falcon console

The **console** is the first-party web application for end users and admins who use Falcon day to day. It uses the **same Falcon Auth server** as your integrated products (default setup).

Topics

Document	Description
Account, profile, and connected apps	User profile and Auth app directory
Organizations	Creating orgs and switching context
Connections	Viewing and managing Connect relationships

What the console is not

- It is **not** a full “Falcon Auth admin” or “Connect catalog CMS” for every database table. Registering new `falcon_auth_app` or `falcon_app` rows is typically an **operator** task (SQL, migrations, internal tooling).
- In-product **SDK documentation** may appear inside the console for developers; see SDK docs surface.

Related reading

- Architecture
- Falcon Auth organizations

Registering Falcon Auth client apps

Every application that uses Falcon Auth as its identity provider must be registered in the database table `falcon_auth_app`. Registration is **operator-driven** (SQL seed, migration, or admin tooling); there is no requirement for a self-service developer portal in the default stack.

What a registration defines

Field (logical)	Purpose
id	Stable primary key for the row (text).
name	Human-readable name (shown on hosted sign-in).
publishable_key	Public identifier, conventionally `pk_…`. Embedded in frontend config and sent as `X-Falcon-App-Id`.
allowed_origins	JSON array of origins (for example `["https://app.example.com"]`) that may call the auth API with this key. Used for CORS on the auth server and for Better Auth **trusted origins** when this app is in play.
redirect_urls	JSON array of **full URLs** allowed as `redirect_uri` for **hosted** sign-in and sign-up. Matching is **exact**.
secret_key_hash	Optional server-side secret storage for future flows (may be null in demos).
settings	Optional JSON for future per-app configuration.

Linking users to apps

When requests include `X-Falcon-App-Id`, the auth layer can associate sign-ups and sessions with the corresponding `falcon_auth_app` so you know **which products** a user has used. The console's "connected apps" style UI reads this association.

Relationship to Falcon Connect apps

Important: `falcon_auth_app` and `falcon_app` (Connect) are different tables.

- **Auth app** — "This SPA or mobile shell uses Falcon to sign users in."
- **Connect app** — "This product participates in installation requests and connections."

A single product in your portfolio might have **both** rows (same company branding, different ids and keys). Demo projects use distinct ids for source/target Connect apps and separate Auth publishable keys.

Example seed snippet

Your repository may ship SQL similar to:

```
INSERT INTO falcon_auth_app (id, name, allowed_origins, redirect_urls
VALUES (
  'my-app-auth',
  'My Customer Portal',
  '["https://portal.example.com"]'::jsonb,
  '["https://portal.example.com/after-login"]'::jsonb,
  'pk_live_abc123',
  …
);
```

Adjust origins and redirect URLs for each environment (development, staging, production).

Checklist for a new SPA

1. Create `falcon_auth_app` row with **publishable key** you will put in frontend env (for example `VITE_FALCON_PUBLISHABLE_KEY`).
2. Add **every browser origin** your SPA uses to `allowed_origins` (local dev ports count).
3. Add **every post-login URL** you pass as `redirect_uri` to `redirect_urls`.
4. Point `VITE_FALCON_AUTH_URL` (or equivalent) at your auth server's public base URL.
5. Ensure **CORS** on the auth server allows your origin when the publishable key header is present (handled automatically when registration is

correct).

Related topics

- Centralized sign-in
- Sessions and cookies
- Environment variables

Centralized sign-in (hosted UI)

Falcon Auth supports a **hosted** sign-in and sign-up experience on the **auth server's origin**, similar in user experience to centralized providers like Clerk: the user leaves your application briefly, authenticates on Falcon, then returns to your app already signed in.

Why use hosted sign-in?

- **Consistent branding and UX** for authentication on one domain you control.
- **Clear security boundary**: credentials are submitted directly to the auth server origin, which simplifies reasoning about cookies and CSRF for the login step.
- **Alignment with allow lists**: each return URL is explicitly registered.

Embedded forms in your own SPA are still possible via the SDK, but many teams prefer the redirect model for production.

User-visible flow

1. The user clicks "Sign in" (or similar) in your app.
2. Your app sends the browser to a URL on the auth server:
   - Sign-in: `GET /hosted/sign-in`
   - Sign-up: `GET /hosted/sign-up`
3. Query parameters:
   - `client_id` — your app's **publishable key** (for example `pk_demo_source`). This identifies the Falcon Auth client registration.
   - `redirect_uri` — a **full URL** (with `http` or `https`) where the user should land after success. Example: `https://app.example.com/dashboard`.
4. The auth server checks that:
   - `client_id` resolves to a registered `falcon_auth_app` row.
   - `redirect_uri` is **exactly** listed in that row's `redirect_urls` JSON array (open-redirect protection).

5. The user enters email and password on the hosted page. The page calls the normal Better Auth endpoints (`/api/auth/sign-in/email` or `/api/auth/sign-up/email`) with:
   - Header `X-Falcon-App-Id` with your publishable key as the value, so Falcon can link the user to your app and apply trusted origins.
   - Body field `callbackURL` set to `redirect_uri` for sign-in (Better Auth uses this to signal a browser redirect in the JSON response).
6. After sign-in, the browser navigates to `redirect_uri`. Your app loads and the SDK (or your code) calls `get-session` on the auth server with **credentials included**; the session cookie was set on the auth domain during login, so the user is recognized.

Building the hosted URL in your app

Use the SDK helpers (recommended):

- `buildFalconHostedSignInUrl(config, { redirectUri })`
- `buildFalconHostedSignUpUrl(config, { redirectUri })`

See Hosted sign-in URLs.

Registration requirements

For hosted sign-in to succeed:

1. `allowed_origins` on your `falcon_auth_app` must include your app's **origin** (scheme + host + port), so Better Auth accepts your `callbackURL` as a trusted redirect target.
2. `redirect_urls` must include each `redirect_uri` you pass to `/hosted/sign-in` or `/hosted/sign-up`, **character-for-character** (including trailing slashes if used).

Example: if you redirect to `http://localhost:3010/dashboard`, the database must list exactly `http://localhost:3010/dashboard`, not only `http://localhost:3010`.

Hosted sign-up behavior

Sign-up via Better Auth returns a session token in JSON when email verification is not blocking auto sign-in. The hosted page redirects the

browser to `redirect_uri` after a successful sign-up response. Ensure that URL is allowlisted the same way as for sign-in.

Troubleshooting

Symptom	Things to check
“Unknown client_id”	Publishable key mismatch; row missing in `falcon_auth_app`.
“redirect_uri is not allowed”	Add exact URL to `redirect_urls`; re-run seed or migrate.
“Invalid callbackURL” from API	`allowed_origins` missing the **origin** of `redirect_uri`.
User returns to app but appears signed out	SDK not using `credentials: 'include'`; CORS on auth server blocking your origin; third-party cookie policies in unusual browser setups.

Related topics

- App registration
- Sessions and cookies
- Architecture

Organizations (Better Auth plugin)

Falcon's console and demos use Better Auth's **organization** plugin. Organizations give you:

- **Tenant-style grouping** for users (membership with a **role**).
- An **active organization** selection in the client, used when calling APIs that require `X-Organization-Id` (Falcon Connect).

Roles

Membership rows carry a **role** string (for example `owner`, `admin`, `member`). Falcon Connect uses this role to decide whether a principal may **create installation requests**, **approve** them, **revoke** or **pause** connections, and run **scope checks**. See Permissions matrix.

Creating an organization

In the console (or any client using the same auth server), users with access to the organization APIs can **create** an organization with a name and slug. After creation, the client typically **sets the active organization** so subsequent operations are scoped correctly.

Demo applications often **force** users through an **org creation** route when they have zero memberships, because Connect calls without a known organization are ambiguous.

Organizations vs Connect `organizationId`

Connect stores `organization_id` on installation requests and connections. That value must match the **Better Auth organization id** your client sends as `X-Organization-Id` so the Connect service can load the caller's membership and enforce permissions.

Multi-org users

Users may belong to multiple organizations. The **active org** in the UI (console or your app) determines which Connect data they see and which org id is sent on API requests. Switching org in the console switches this context.

Related topics

- Organizations in the console
- Falcon Connect concepts
- Calling the Connect API

Falcon Auth

Falcon Auth is your deployment's **identity provider**: users authenticate against the auth server, and integrated applications use the SDK (or raw HTTP) to obtain session state and optional organization context.

Topics

Document	Description
Centralized sign-in	Hosted pages, `client_id`, `redirect_uri`, and return to your app
Registering client apps	`falcon_auth_app`, origins, redirect URLs, publishable keys
Sessions and cookies	Cross-origin session behavior and `get-session`
Organizations	Better Auth organization plugin and membership

Relationship to Falcon Connect

Connect uses the same user identities. It validates the caller by session cookie or JWT, then checks **organization membership** for `X-Organization-Id`. See Calling the Connect API.

SDK entry points

- React: React integration
- Hosted URL builders: Hosted sign-in URLs
- Backend: Server session verification

Sessions and cookies

Falcon Auth builds on **Better Auth**. Sessions are created when a user signs in; the auth server sets **HTTP-only cookies** on the **auth server's domain** (with `SameSite` and `Secure` attributes appropriate for cross-site API usage in typical deployments).

How your SPA sees the session

The Falcon SDK's React client calls endpoints such as `/api/auth/get-session` on the configured `serverUrl`, with `credentials: 'include'`. The browser attaches cookies for the auth host to those requests even when the JavaScript runs on your app's origin, **provided** CORS allows your origin and credentials.

That means:

- The user does **not** need a session cookie on *your* app's domain for the default SDK flow.
- Your app's origin must be listed in the Auth app's `allowed_origins` and the auth server's CORS rules must reflect that.

Centralized sign-in and cookies

When the user completes hosted sign-in on the auth origin, the session cookie is set in that context. When they are redirected back to your app, subsequent `get-session` calls from your origin still send that cookie to the auth host. No extra step is required in the happy path.

Console and your apps

If the console and your product both use the **same auth server** and the **same real user accounts**, a single sign-in can cover both experiences from the browser's perspective (same auth cookie jar). Organization and Connect data still depend on **which org** is active and which **Connect apps** exist.

Server-side verification

Backend services that receive the browser's `Cookie` header can forward it to `get-session` or use JWT verification. See Server session verification.

JWT option (Connect)

The Connect service can authenticate some calls using a **Bearer** token verified against the auth server's **JWKS** (`/.well-known/jwks.json`). That path is useful for machine-to-machine or mobile patterns. Browser demos often rely on **cookies** only.

Details: Calling the Connect API.

Operational tips

- Keep **clock skew** minimal when using JWTs.
- When rotating `BETTER_AUTH_SECRET`, plan session invalidation carefully (Better Auth behavior applies).
- Prefer **HTTPS** everywhere in production; `Secure` cookies require it.

Related topics

- Centralized sign-in
- Architecture

Calling the Connect API

All Connect endpoints are scoped to an **organization** and require a known **authenticated user** with **membership** in that organization.

Required header: `X-Organization-Id`

Every request must include:

```
X-Organization-Id: <better-auth-organization-id>
```

The value must be the **organization id** from Better Auth's organization plugin (the same id the console uses as "active org").

If this header is missing or empty, the service responds with **401** rather than guessing an organization. This avoids accidental data leaks from "first membership wins" behavior.

How the Connect service authenticates the user

The Connect runtime resolves a **principal**:

1. **Session cookie** — If the `Cookie` header is present, Connect proxies to the auth server's `/api/auth/get-session` using those cookies. If a user is returned, Connect loads `member` for `(userId, organizationId)`.
2. **Bearer JWT** — If `Authorization: Bearer …` is present, Connect verifies the token using the auth server's **JWKS** (`/.well-known/jwks.json`), reads `sub` as the user id, then loads membership the same way.

If neither strategy yields a valid user bound to the requested organization, the request is **unauthorized**.

CORS and credentials

Browser apps must call Connect with `credentials: 'include'` when using cookie-based sessions, and the Connect service must allow your origin via its

CORS configuration (often a comma-separated allow list including console and app URLs).

Roles after authentication

Once the principal is known, Connect checks **role** strings (`owner`, `admin`, `member`) before allowing sensitive mutations. See Permissions matrix.

Example: fetch from a browser

```
const res = await fetch(`${connectBase}/v1/connections`, {
  credentials: "include",
  headers: {
    "X-Organization-Id": activeOrganizationId,
  },
});
```

Example: server-to-server

Prefer **JWT** or a dedicated service account pattern if you introduce one; the stock stack documents JWT verification against Falcon Auth's JWKS for Connect.

Reliability note

The implementation may **retry once** when cookies and organization are present to smooth rare races between session establishment and membership reads. Clients should still treat **401** as "user must sign in or pick an org."

Related topics

- Architecture
- Server session verification
- Organizations

Falcon Connect concepts

Falcon Connect app (`falcon_app`)

A **Connect app** is a registered product in the Connect catalog. It has:

- **id** — stable primary key used in APIs (`sourceAppId`, `targetAppId`).
- **slug** — unique short identifier (often used in URLs or logs).
- **name** and **description** — human-facing labels.
- **status** — typically `active` or `inactive` for catalog filtering.

Connect apps are **independent** of `falcon_auth_app` rows. In real deployments you might create both for one customer-facing product, but they serve different subsystems.

Capabilities and scopes

Each Connect app may declare **capabilities** in `app_capability`:

- **scope_key** — string identifier (for example `customers.read`, `demo.read`).
- **description** — optional explanation for admins and UIs.

When a **connection** is formed, a subset of these keys is **granted** and stored (for example in `connection_scope`). Runtime authorization in *your* services should treat granted scope keys as the contract of what the source app may request.

Organization scope

Almost all Connect resources are tied to an `organization_id`:

- **Installation requests** belong to one org.
- **Connections** belong to one org.

The Connect API expects the caller to specify which organization they act within using the `X-Organization-Id` header. The service then loads the

caller's **membership** for that org and derives their **role** for permission checks.

Installation request

An **installation request** records intent to connect:

- **source_app_id** — the app initiating access (the "installer" or hub).
- **target_app_id** — the app that will expose data or actions.
- **requested_scopes** — array of scope keys the source wants.
- **status** — `pending`, `approved`, `rejected`, or `expired`.
- **initiated_by_user_id** — who created the request.

Creation is allowed for **owner**, **admin**, and **member** roles by default. Approval is restricted to **owner** and **admin**.

Connection

A **connection** is the durable outcome after approval (or equivalent automation):

- Links the same **organization**, **source**, and **target** apps.
- Carries **status**: `active`, `paused`, or `revoked`.
- Holds **granted scopes** and optional **settings** snapshots for auditing and product behavior.

Paused connections retain history but should not be used for new operational traffic until **resumed**. Revoked connections are terminal from a permission perspective.

Settings and audit

The schema supports **per-connection settings** and **audit logs** for traceability (who approved, who revoked, and so on). Product UIs and operators can surface these for compliance and debugging.

Related topics

- Installation requests and approval
- Managing connections
- Permissions matrix

Installation requests and approval

This page describes the **end-to-end collaboration pattern** Falcon Connect supports: one application (the **source**) asks to connect to another (**target**), and an authorized user on the target side **approves** the request, resulting in a **connection**.

Preconditions

1. **Identity**: the user is signed in with Falcon Auth (session cookie or JWT when calling APIs).
2. **Organization**: the user is a **member** of an organization, and clients send its id as `X-Organization-Id`.
3. **Catalog**: both apps exist as `falcon_app` rows, and the target exposes **capabilities** the source may request.
4. **Roles**:
   - **Creating** a request: `owner`, `admin`, or `member`.
   - **Approving** a request: `owner` or `admin` only.

See Permissions matrix.

API steps (logical)

1. Discover capabilities (optional but typical)

`GET /v1/apps/{targetAppId}/capabilities`

Returns which **scope_key** values exist for the target. A source app UI often uses this to populate checkboxes or to default “request all read scopes.”

2. Create installation request

`POST /v1/installation-requests`

Body includes `sourceAppId`, `targetAppId`, `requestedScopes`, and optional `settingsDraft`.

The service rejects duplicates if a pending request or non-revoked connection already exists for the same org and app pair (conflict semantics).

3. Approve

`POST /v1/installation-requests/{requestId}/approve`

Only a principal with approve rights can succeed. Success creates a **connection** and records granted scopes.

Browser handoff pattern (demos)

Real products often **redirect the user's browser** from the source app to the target app so the approver is in the right UX context (and may need to sign in or create an org on the target side).

A typical pattern:

1. Source app calls `POST /v1/installation-requests` via XHR/fetch with credentials.
2. Source app navigates to a **target URL** with query parameters such as `requestId` and a `returnUrl` back to the source.
3. Target app loads pending requests (or the specific id), shows details, and offers **Approve**.
4. On approve, target app calls `POST /v1/installation-requests/{id}/approve`, then sets `window.location` to `returnUrl`.

Security product note: `returnUrl` should be **validated** so you only redirect to origins you trust (demos compare origins to a configured peer origin).

Same user, same org across apps

For the simplest SSO story, the **same Falcon user** should complete both sides, and the **organization id** used in `X-Organization-Id` should be the **same Better Auth organization** on both apps. Otherwise the installation request row may not appear where the approver expects, or approval may fail permission checks.

After approval

- The **source** can list **connections** (`GET /v1/connections`) and show the new link.

- Either side (subject to role) may **pause**, **resume**, or **revoke** via connection endpoints. See Managing connections.

Related topics

- Concepts
- Calling the Connect API
- Connections in the console

Managing connections

Once a **connection** exists between a **source** and **target** app for an **organization**, operators and product UIs can manage its lifecycle through the Connect API (and mirrored features in the console when enabled).

Listing connections

`GET /v1/connections`

Returns summaries for the organization implied by `X-Organization-Id`. UIs often group rows by **status** (`active`, `paused`, `revoked`).

Connection detail

`GET /v1/connections/{connectionId}`

Returns richer information including **granted scopes** and optional **settings** payload useful for support and auditing.

Pause and resume

- `POST /v1/connections/{connectionId}/pause` — temporarily disables active use; implementation should treat paused connections as non-operational for new work.
- `POST /v1/connections/{connectionId}/resume` — restores an active state from paused.

Roles: `owner` and `admin` only (see Permissions matrix).

Revoke

`POST /v1/connections/{connectionId}/revoke`

Marks the connection **revoked**. This is the strongest “turn off access” lever; restoring usually requires a new installation flow rather than resume.

Trigger sync (optional integration hook)

`POST /v1/connections/{connectionId}/sync`

Creates a **sync job** record. Whether and how background workers process jobs depends on your deployment; the API models the **request** side for product consistency.

Scope check

`POST /v1/scope-check`

Body identifies a **connection**, an **app id**, and a **scope** string. Response includes whether access is **granted** for that principal under current connection state.

Useful for **defense in depth**: even if your gateway trusts “there is a connection,” individual endpoints can re-verify a specific scope.

Console parity

The web console includes connection list and detail routes that call the same API patterns: tabs by status, detail view with actions (revoke, pause, resume, sync) subject to the user’s role.

See Connections in the console.

Related topics

- Concepts
- Installation requests and approval

Falcon Connect

Falcon Connect manages **relationships between registered applications** within an **organization**: which app may integrate with which, under which **scopes**, and whether access is **pending**, **active**, **paused**, or **revoked**.

Topics

Document	Description
Concepts	Apps, capabilities, scopes, org scope
Installation requests and approval	Source/target flow and browser handoff
Managing connections	Pause, resume, revoke, sync, scope check
Calling the Connect API	`X-Organization-Id`, cookies, JWT

Who should read this

- **Product engineers** designing cross-app integrations.
- **Frontend developers** driving installation UIs.
- **Backend operators** securing Connect behind the same identity stack as Falcon Auth.

SDK helpers

Optional display utilities live in `@falcon-framework/sdk/connect`. See Connect display helpers.

Architecture

This page describes how Falcon's pieces fit together at runtime. Exact hostnames and ports depend on your environment; local development often uses separate ports for each service.

Logical components

```
flowchart LR
  subgraph Browser
    SPA[Your SPA / Console]
  end
  subgraph Falcon
    AUTH[Auth server]
    CONN[Connect service]
  end
  DB[(Postgres)]
  SPA ⟶|session API + hosted sign-in| AUTH
  SPA ⟶|Connect API + org header| CONN
  AUTH ⟶ DB
  CONN ⟶ DB
  CONN ⟶|get-session or JWKS| AUTH
```

Auth server

- Serves **Better Auth** routes (for example `/api/auth/sign-in/email`, `/api/auth/get-session`, organization APIs used by the console).
- Serves **hosted** HTML flows at `/hosted/sign-in` and `/hosted/sign-up` for centralized sign-in (see Centralized sign-in).
- Applies **dynamic CORS**: the console origin is always allowed; other origins may be allowed when a valid `X-Falcon-App-Id` (publishable key) matches a registered app whose `allowed_origins` includes the request `Origin`.
- Exposes **user app directory** routes (for example listing apps the user has used) for the console.

Connect service

- Serves the **Falcon Connect HTTP API** (for example `/v1/apps`, `/v1/installation-requests`, `/v1/connections`).
- **Does not** implement its own username/password login. Instead it establishes **who** is calling by:
  - Forwarding **cookies** to the auth server's `get-session`, or
  - Validating a **Bearer JWT** against the auth server's JWKS.
- Requires an `X-Organization-Id` header on requests so every operation is scoped to one organization and membership can be checked.

Console

- A SPA that uses the **same auth server** as your apps (no separate "console-only" identity product in the default setup).
- Uses Better Auth's **organization** plugin for creating orgs, switching active org, and membership.
- When Connect is configured, uses the Connect API with the user's session cookies and the **active organization** id.

Your applications

- Browser apps load the SDK, configure `serverUrl` and `publishableKey`, and either redirect to **hosted** sign-in or use optional **embedded** SDK forms.
- Server apps can verify sessions with `verifySession`.

Data storage

A single **PostgreSQL** database typically holds:

- **Auth tables** (Better Auth: users, sessions, accounts, verification, organization plugin tables).
- `falcon_auth_app` — registered Auth clients (origins, redirect URLs, publishable key).
- `app_user` — links users to Auth apps they have used.
- **Connect tables** — `falcon_app`, `app_capability`, `installation_request`, `connection`, `connection_scope`, settings, audit, and related rows.

The auth server and connect service are configured with database access to these schemas.

Request flows (high level)

Sign-in (centralized)

1. User opens your app; your app redirects to `https://your-auth-server/hosted/sign-in?client_id=…&redirect_uri=…`.
2. User submits credentials on the auth origin; the auth server sets a **session cookie** for its domain.
3. Browser navigates to `redirect_uri` on your app's origin.
4. Your app's SDK calls `get-session` on the auth server with `credentials: 'include'` so the cookie is sent; the user appears signed in.

Details: Centralized sign-in.

Connect: create installation request

1. Authenticated user (session cookie or JWT) calls `POST /v1/installation-requests` with `X-Organization-Id` set.
2. Connect resolves the principal (user id + org id + **role** from membership).
3. If the role may create requests, a row is created in `installation_request` with status `pending`.

Connect: approve and form a connection

1. A user with **owner** or **admin** role on the same organization (typically on the **target** side in product flows) calls `POST /v1/installation-requests/:id/approve`.
2. Connect creates a **connection** and records granted **scopes**.

Details: Installation requests and approval.

Security notes (operator view)

- **Allowed origins** for each Auth app limit which websites may call the auth API with that publishable key.
- **Redirect URLs** for hosted sign-in are an **exact-match** allow list to prevent open redirects.
- Connect requires **explicit organization** on every call to avoid ambiguous tenancy.

- **Roles** gate create vs approve vs revoke/pause operations; see Permissions matrix.

Related reading

- Environment variables
- Calling the Connect API

Getting started

Introduction to Falcon for new readers.

Document	Description
What is Falcon?	Products, audiences, terminology
Architecture	Services, database, and request flows

Next: Falcon Auth or Falcon Connect.

What is Falcon?

Falcon is a self-hosted platform for two closely related problems:

1. **Falcon Auth** — A centralized identity service (similar in *shape* to Clerk or Auth0's hosted experience). Your applications send users to Falcon to sign in or sign up; after authentication, users return to your app with a session that is anchored on the auth service.
2. **Falcon Connect** — A connection layer between **registered applications** inside an **organization**. It models which app may access which other app, under which **scopes** (capabilities), and supports an explicit **request and approval** flow before access is granted.

You typically run:

- An **auth server** (Falcon Auth HTTP API and hosted sign-in pages).
- A **connect service** (Falcon Connect HTTP API).
- A **console** web app where users manage their profile, see which apps they have used, work with **organizations**, and inspect **connections**.

Client applications embed the **Falcon SDK** (for React and server helpers) and configure a **publishable key** so Falcon knows which registered app is calling.

Who this guide is for

Audience	What you need from Falcon
End users	Sign in once with Falcon, use integrated products, manage account and orgs in the console.
Integrators / frontend developers	Add sign-in, session checks, and (optionally) Connect API calls from your app.
Backend developers	Verify sessions, call Connect with correct headers, enforce org and role rules in your own APIs.
Operators / platform admins	Deploy services, configure CORS and URLs, seed or migrate database rows for apps and

Audience	What you need from Falcon
	capabilities.

Core terminology

Term	Meaning
Auth server	HTTP service exposing Better Auth–compatible routes (for example `/api/auth/*`) and Falcon's hosted sign-in UI (`/hosted/sign-in`, `/hosted/sign-up`).
Connect service	HTTP service exposing the Falcon Connect API under a `/v1` prefix.
Console	First-party web UI for account, orgs, connected auth apps, and (when configured) connections.
Publishable key (`pk_…`)	Public identifier for a **Falcon Auth client app**. Sent on requests (for example `X-Falcon-App-Id`) so the auth server can apply the right CORS, trusted origins, and user–app linking.
Falcon Auth app (`falcon_auth_app`)	A row registering a client application: allowed browser origins, allowed post-login redirect URLs, publishable key.
Falcon Connect app (`falcon_app`)	A product registered for Connect: slug, name, status, and linked **capabilities** (scopes).
Organization	A tenant boundary for Connect (and org membership in Auth). Connect data is scoped by organization.
Installation request	A proposal to connect a **source** app to a **target** app with a set of requested scopes, pending approval.
Connection	An approved, durable link between two apps for one organization, with granted scopes and lifecycle (active, paused, revoked).

Term	Meaning
Scope / capability	A string key (for example `demo.read`) declared on a target app; connections grant a subset of these to a source app.

Falcon Auth vs Falcon Connect

- **Auth** answers: *Who is this user?* Sessions and users live in the auth database; cookies for the session are set on the auth server's origin (with appropriate cross-site settings for embedded API calls from your app's origin).
- **Connect** answers: *May this organization's installation allow app A to act on app B under scope S?* It does not replace Auth; it relies on knowing the **user** and **organization** for each API call.

Together they support patterns such as: sign in with Falcon, pick an organization, then create or approve a connection between two products your company uses.

Next steps

- Architecture — how services talk to each other and to the database.
- Falcon Auth overview or Falcon Connect overview depending on your goal.

Environment variables (reference)

Exact names depend on your deployment (Cloudflare Workers bindings, Vite `import.meta.env`, etc.). This page lists **conceptual** variables used across the default apps in this repository.

Auth server

Variable	Role
`DATABASE_URL`	PostgreSQL connection for Better Auth and Falcon tables.
`BETTER_AUTH_SECRET`	Signing secret for Better Auth.
`BETTER_AUTH_URL`	Public base URL of the auth server (used as Better Auth `baseURL` and to derive trusted auth origin).
`CORS_ORIGIN`	Allowed origin(s) for the console and related first-party apps; may be comma-separated.

Connect service

Variable	Role
`DATABASE_URL`	Same Postgres; Connect uses connection schema tables.
`BETTER_AUTH_URL`	Used to call `get-session` and JWKS for principal resolution.
`CORS_ORIGIN`	Comma-separated allowed browser origins for the Connect API.

Console (Vite)

Typical `VITE_*` values include:

Variable	Role
`VITE_SERVER_URL`	Falcon Auth server URL used by the console SPA.
Other app-specific URLs	Connect base URL if the console exposes connection management.

Demo applications (Vite)

Demos use variables such as:

Variable	Role
`VITE_FALCON_AUTH_URL`	Auth server base URL.
`VITE_FALCON_PUBLISHABLE_KEY`	Publishable key for the demo's `falcon_auth_app` row.
`VITE_CONNECT_URL`	Connect service base (without `/v1`).
`VITE_FALCON_APP_ID` / `VITE_PEER_APP_ID`	Connect `falcon_app` ids for source/target demos.
`VITE_PEER_APP_ORIGIN`	Trusted peer origin for `returnUrl` validation in cross-app redirects.
`VITE_APP_PUBLIC_ORIGIN`	This demo's public origin (used to build hosted `redirect_uri` values).
`VITE_FALCON_CONSOLE_URL`	Link out to the console.

See each app's `.env.example` for the authoritative list.

Related topics

- Architecture
- App registration

Connect permissions by organization role

Falcon Connect enforces **organization-scoped** permissions using the caller's **membership role** (Better Auth organization plugin). The checks live in `@falcon-framework/auth` permission helpers used by Connect services.

Matrix

Action	`owner`	`admin`	`member`
Create installation request	yes	yes	yes
Approve installation request	yes	yes	no
Revoke connection	yes	yes	no
Pause connection	yes	yes	no
Resume connection	yes	yes	no
Trigger sync	yes	yes	no
Scope check	yes	yes	yes

Notes

- **Approve** is the gate that turns a **pending installation request** into a **connection**. Product flows should route that action to trusted administrators on the **target** side (or your policy equivalent).
- **Member** can initiate integration work but cannot tear down or pause infrastructure-level links—reducing accidental outages.
- If you introduce custom roles, you must update **both** Better Auth membership data and the permission functions to stay coherent.

Related topics

- Concepts

- Organizations in the console

Reference

Quick-reference material for operators and integrators.

Document	Description
Environment variables	Common bindings by service
Permissions matrix	Connect capabilities by org role

Source of truth

Behavior may evolve with the codebase. When in doubt, prefer:

- Auth: Better Auth route definitions and Falcon-specific middleware in the auth server.
- Connect: `FalconConnectionApi` schema in the `@falcon-framework/connection` package.
- Database: Drizzle schema under `@falcon-framework/db/schema`.

Connect display helpers

The entry `@falcon-framework/sdk/connect` exposes small **pure helpers** for turning raw Connect API responses into display-friendly structures (labels, sorted lists). They do **not** call HTTP themselves.

Typical use

1. Your UI fetches `/v1/apps` and `/v1/connections` with credentials and `X-Organization-Id`.
2. You pass those results into helpers such as `resolveFalconConnectionsDisplay` to align connection rows with app names from the directory.

Why it exists

Product screens often need a **single list model** for rendering cards: connection id, status, human-readable source/target names, and deep links. Keeping that mapping in the SDK reduces duplication between demos, console code paths, and customer apps.

Related topics

- Falcon Connect concepts
- Managing connections

Hosted sign-in URL helpers

When using the **centralized** Falcon Auth experience, your application should send users to the auth server's hosted pages with the correct query string.

Functions

```
import {
  buildFalconHostedSignInUrl,
  buildFalconHostedSignUpUrl,
} from "@falcon-framework/sdk/react";
// or: "@falcon-framework/sdk"
```

`buildFalconHostedSignInUrl(config, { redirectUri })`

Returns an absolute URL:

- Path: `/hosted/sign-in`
- Query: `client_id=<publishableKey>&redirect_uri=<encoded redirectUri>`

`buildFalconHostedSignUpUrl(config, { redirectUri })`

Same pattern for `/hosted/sign-up`.

Choosing `redirectUri`

- Must be a **full URL** (`http://localhost:3010/dashboard`, `https://app.example.com/onboarding/done`).
- Must appear **exactly** in your `falcon_auth_app.redirect_urls` array.
- Its **origin** must be allowed in `allowed_origins` so Better Auth accepts it as `callbackURL` on sign-in.

Typical usage patterns

Full-page navigation (recommended)

```
window.location.href = buildFalconHostedSignInUrl(config, {
  redirectUri: `${appPublicOrigin}/dashboard`,
});
```

Router-friendly landing route

A `/sign-in` route in your SPA can immediately redirect to the hosted URL after checking `useFalconAuth().isSignedIn` to skip unnecessary round trips.

Related topics

- Centralized sign-in
- App registration

React: provider, hooks, and components

Configuration object

The SDK expects:

- `serverUrl` — public base URL of the Falcon Auth server (no trailing slash required; normalize consistently).
- `publishableKey` — value of `X-Falcon-App-Id` for your `falcon_auth_app` row.

```
const config = {
  serverUrl: "https://auth.example.com",
  publishableKey: "pk_live_abc123",
};
```

`FalconAuthProvider`

Wrap your tree so hooks and components can access the Better Auth–compatible client:

```
import { FalconAuthProvider } from "@falcon-framework/sdk/react";

export function App() {
  return (
    <FalconAuthProvider config={config}>
      <Routes />
    </FalconAuthProvider>
  );
}
```

Hooks

`useFalconAuth()`

Returns:

- `user`, `session` — from `useSession()` under the hood.
- `isLoaded`, `isSignedIn` — derived readiness flags.
- `signOut` — Better Auth sign-out function.
- `client` — raw client for advanced calls.

`useUser()` / `useSession()`

Convenience subsets of the same state.

Optional UI components

The SDK ships **SignIn** and **SignUp** forms that POST through the configured client (embedded UX on your origin). Many production apps instead use hosted sign-in.

UserButton renders a compact account control with sign-out, suitable for headers.

Better Auth client behavior

The Falcon client injects `X-Falcon-App-Id` on fetch calls automatically so the auth server can resolve your `falcon_auth_app` registration.

Default fetch plugins include Better Auth's **redirect** helper: when an endpoint returns `{ redirect: true, url }`, the client may set `window.location` automatically (relevant for some flows with `callbackURL`).

Organizations in your SPA

The stock SDK docs focus on **session**. Organization **create / list / setActive** APIs come from Better Auth's **organization client plugin** if you add it the same way the console does. Demos combine Falcon Auth sessions with org-aware Connect calls.

See Organizations.

Related topics

- Centralized sign-in

- Sessions and cookies

Falcon JavaScript SDK

The package `@falcon-framework/sdk` helps React and Node (or compatible) runtimes integrate Falcon Auth and optional Connect UI helpers.

Topics

Document	Description
React integration	Provider, hooks, optional components
Hosted sign-in URLs	`buildFalconHostedSignInUrl` / `buildFalconHostedSignUpUrl`
Server session verification	`verifySession`
Connect display helpers	`@falcon-framework/sdk/connect`

Package entry points

Import path	Purpose
`@falcon-framework/sdk`	Core client factory and hosted URL helpers
`@falcon-framework/sdk/react`	React provider, hooks, UI primitives
`@falcon-framework/sdk/server`	Session verification for backends
`@falcon-framework/sdk/connect`	Optional connection list display utilities

Requirements (React UI)

Pre-built components expect **React 18+** and **Tailwind CSS** with shadcn-style CSS variables, matching the Falcon design tokens used in demos.

Related reading

- Falcon Auth overview
- Falcon Connect overview

Server session verification

The `verifySession` helper (from `@falcon-framework/sdk/server`) lets your **backend** confirm that an incoming HTTP request belongs to a signed-in Falcon user.

Behavior

Given:

- `serverUrl` — Falcon Auth base URL.
- `publishableKey` — your app's publishable key.
- An incoming **Request** (or headers object) that may include a `Cookie` header.

The helper forwards cookies to `GET /api/auth/get-session` on the auth server, optionally with `X-Falcon-App-Id`, and parses the JSON payload.

Returns `null` when there is no valid session; otherwise returns **user** and **session** objects mirroring Better Auth's shape.

When to use it

- **BFF / API routes** in your own server that must not trust client-side session flags alone.
- **Bridge services** that need a stable user id before talking to your domain databases.

Limitations

- This verifies **Falcon Auth session**, not Falcon Connect organization membership. Connect adds its own `X-Organization-Id` and membership checks.
- For machine clients, consider **JWT** verification against JWKS instead of cookies.

Related topics

- Sessions and cookies
- Calling the Connect API